Parallel post-training quantization: lessons and build plan

Oct 31, 2026 · @Bryan PALOMO

Overview

This project parallelizes two post-training quantization (PTQ) algorithms on a small CNN: **KL-divergence calibration** picks INT8 scales for activations, and **GPTQ** turns FP32 weights into INT4 with error compensation. Each one is built twice, in CUDA and in MPI, and compared on time, speedup and accuracy retained.

Together they produce a **W4A8** model: 4-bit weights, 8-bit activations. The test model is ResNet-20 on CIFAR-10, with larger ResNet-18 layer shapes used for the scaling experiments.

Program	Algorithm	Platform	What makes it parallel
P1	KL calibration (+ MSE, percentile variants)	CUDA	Histograms built from about 100M activations; about 1,900 candidate thresholds scored at once
P2	KL calibration (+ variants)	MPI	Calibration images split across ranks; histograms merged with one allreduce; candidates split across ranks
P3	GPTQ	CUDA	Hessian X^TX is a large matrix multiply; the column sweep is sequential across columns but parallel across rows
P4	GPTQ	MPI	Partial Hessians per rank, summed with an allreduce; weight rows split across ranks

How to use this doc

1. Read Part 1 in order. Each lesson builds on the one before and ends with self-check questions. Budget about 2 hours per lesson, and more for Lessons 4 and 7.

2. Read Part 2 with the lessons fresh. It turns each idea into kernels, data structures and MPI calls, with code skeletons to start from.
3. Follow the study and build plan at the end. It orders the work so each program can be checked against a simpler one before you time anything.

Flags

- The course handout lists Oct 29 for the first algorithm and Nov 25 for the second. Confirm your actual first deadline with the instructor.
- Assumed hardware: one NVIDIA GPU for P1 and P3, and a multi-core machine or small cluster with MPI for P2 and P4.

Part 1: Prior knowledge

Lesson 1: What quantization is

Quantization stores each number as a small integer plus a shared scale. INT8 is 4× smaller than FP32, INT4 is 8× smaller, and integer arithmetic runs faster. Everything later in this doc comes down to one question: which scale to choose.

Why bother

A trained network is mostly numbers stored as 32-bit floats (FP32), 4 bytes each. Running it means streaming those numbers from memory and multiplying them. Memory traffic is usually the bottleneck. Shrinking each number to 8 or 4 bits makes the model smaller, faster to load, and able to use the GPU's integer units, which are faster than its float units.

The catch: an 8-bit integer has only 256 possible values, while a float can represent a huge range very finely. Something gets lost, and quantization is about losing as little as possible.

Floats vs. integers

- **A float** stores a sign, an exponent and a fraction. Its values are dense near zero and sparse far from zero. It can hold both 0.0001 and 10,000.
- **An INT8** stores one of 256 evenly spaced whole numbers, −128 to 127. **INT4** stores one of 16, −8 to 7.

To store a real number in an integer, you choose a **scale** s: the real-world size of one integer step. Divide by s, round, and store the integer. Multiply back by s to recover an approximation.

The formulas

The standard uniform quantizer (Nagel et al., 2021) has a scale s, a zero-point z and a bit-width b:

```
x_int = clamp( round(x / s) + z,  0,  2^b − 1 )
x_hat = s × (x_int − z)
```

Here ⌊·⌉ means round to nearest, clamp forces the result into the integer range, and x̂ is the value you get back.

This project uses the simpler **symmetric** version: z = 0 and a signed range. If the largest value you want to represent is T (the **clipping threshold**), then:

```
s     = T / (2^(b−1) − 1)
x_int = clamp( round(x / s),  −(2^(b−1) − 1),  2^(b−1) − 1 )
x_hat = s × x_int
```

For INT8 that means s = T/127; for INT4, s = T/7. Choosing T is the entire problem of calibration.

Worked example: the two kinds of error

Take four activation values: −0.8, 0.1, 0.35 and 2.9.

Choice A: T = 1.0, so s = 1/127 ≈ 0.00787.

- 0.35 / s = 44.45, which rounds to 44, giving back 0.3465. The error is 0.0035, which is tiny.
- 2.9 / s = 368, which is clamped to 127, giving back 1.0. The error is 1.9, which is huge.

Choice B: T = 2.9, so s = 2.9/127 ≈ 0.02283.

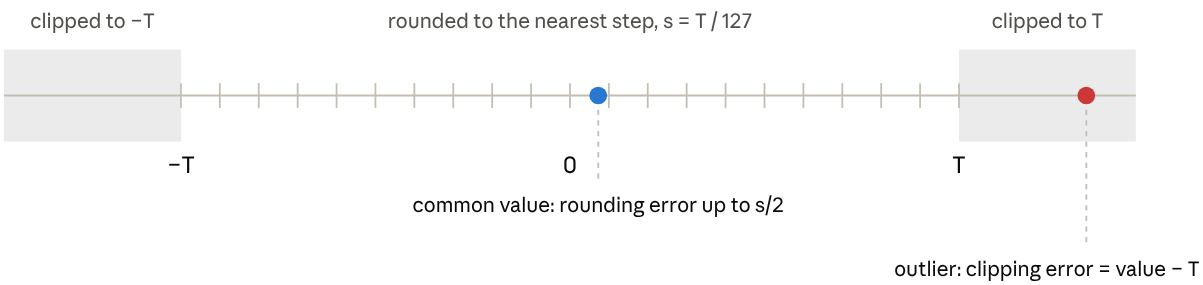
- 0.35 / s = 15.33, which rounds to 15, giving back 0.3425. The error is 0.0075, double choice A's.
- 2.9 comes back exactly, so the error is 0.

That is the two-sided trade-off behind every method in this doc:

- **Rounding error** affects every value inside [−T, T]. It is at most s/2 per value, and averages s²/12 when squared. A larger T means a larger s and more rounding error for everything.
- **Clipping error** affects only values beyond T. A smaller T means more values get clipped, and by more.

Networks are full of values clustered near zero with a few large outliers. Setting T to the maximum protects the rare outliers but blurs all the common values. Setting T too small destroys the outliers, which sometimes matter. Lessons 3 to 5 cover how to choose well.

Inside the range values round to a step; beyond T they are clipped



Raising T shrinks clipping but makes every step coarser; lowering T does the reverse.

symmetric INT8 quantizer · rounding vs clipping

Values between −T and T snap to the nearest of 255 evenly spaced steps; anything beyond T is clamped to ±T.

Why integers make math cheaper

A layer computes sums of products, $y = \Sigma\, w_i\, x_i$. If $w_i \approx s_w \cdot w_{int,i}$ and $x_i \approx s_x \cdot x_{int,i}$, then:

```
y ≈ s_w × s_x × (sum over i of  w_int[i] × x_int[i])
```

The sum is pure integer arithmetic, accumulated in INT32 so it cannot overflow. You multiply by the two scales once, at the end. This only works if every term in the sum shares the same scales. That is why activations get **one scale per tensor**, while weights can get **one scale per output channel**: each output channel is its own sum (Lesson 2).

Fake quantization

To measure accuracy you don't need real integer kernels. You quantize and immediately dequantize ($x \to \hat{x}$) in FP32, then run the normal model. The output is numerically what an integer kernel would produce. Part 2 uses this for every accuracy measurement.

Self-check

1. With T = 6 and INT8, what is s? What integer does 1.5 become, and what value comes back?
2. Why does doubling T roughly double every rounding error?

3. Why can't each activation value have its own scale inside one dot product?

Lesson 2: Where quantization happens in a CNN

Every convolution is secretly a matrix multiply, $Y = X W^T$. The weights W are fixed, so they can be quantized offline (GPTQ). The inputs X change with every image, so their range has to be measured on sample data (calibration).

What a convolution layer computes

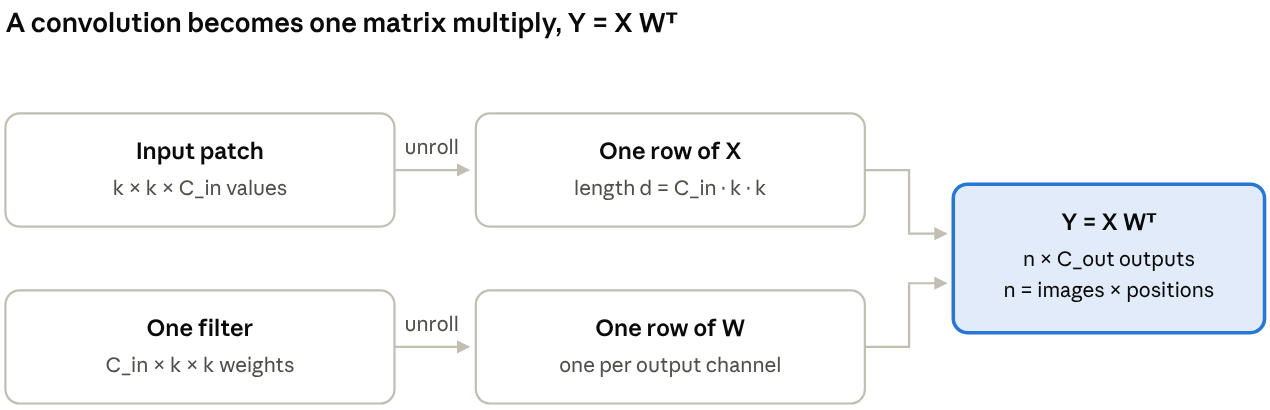
A conv layer takes an input with C_in channels (for example, 16 feature maps of 32×32 pixels). It has C_out filters, each of size C_in × k × k (k = 3 in ResNet). Each output value is the dot product of one filter with one k×k×C_in patch of the input, plus a bias. Slide the filter over every position, repeat for every filter, and you have the output.

Convolution as a matrix multiply (im2col)

The trick that both GPUs and GPTQ rely on:

1. Take every input patch the filter will visit and **unroll it into a row** of length $d = C_{in} \cdot k \cdot k$. Stack one row per output position, for every image. That gives a matrix X with n rows and d columns. This step is called **im2col** (image to columns).
2. **Unroll every filter into a row** of the same length d. Stacking the C_out filters gives the weight matrix W, of shape C_out × d.
3. The whole layer is now one matrix product, $Y = X W^T$, with shape n × C_out.

For a 64-channel 3×3 layer, $d = 64 \cdot 9 = 576$. With 512 calibration images and 8×8 = 64 output positions each, X has n = 32,768 rows. Lesson 6 shows that GPTQ needs exactly this X, so you will implement im2col in Part 2.



Stack one row of X per image and output position (n rows); stack the C_out filters into W.

im2col · convolution as a matrix multiply

Patches become rows of X and filters become rows of W, so the whole layer is one matrix product, which is also the form GPTQ needs.

Weights vs. activations

	Weights	Activations
What they are	The learned filter values (W)	The values flowing between layers (rows of X)
When they are known	Fixed after training	Different for every input image
How the range is found	Look at the weights directly	Run sample images through and record what appears (calibration)
Algorithm in this project	GPTQ, to INT4	KL calibration (with MSE and percentile variants), to INT8
Scale granularity	One scale per output channel (per row of W)	One scale per tensor

Per-channel scales for weights work because each output channel is its own dot product (Lesson 1). They matter a lot in practice. NVIDIA's evaluation found per-tensor weight quantization dropped EfficientNet-b0 to 12.93% accuracy, versus 76.72% with per-channel scales (Wu et al., 2020).

BatchNorm folding

In ResNet, each conv is followed by BatchNorm, which applies a fixed per-channel scale and shift at inference time: $y = \gamma\,(z - \mu)/\sigma + \beta$. Because it is linear, it can be folded into the conv before quantizing:

```
w' = w × γ / σ
b' = β − γ × μ / σ
```

(each per output channel). Always fold first, so you quantize the weights the network actually uses. Folding makes per-channel weight scales close to mandatory, because γ/σ differs wildly between channels.

The test model: ResNet-20 on CIFAR-10

ResNet-20 (He et al., 2015) classifies 32×32 color images into 10 classes. It has about 0.27M weights in 19 conv layers and 1 fully connected layer, in three stages of 16, 32 and 64 channels at 32×32, 16×16 and 8×8 resolution. The largest layer has $d = 576$.

Per image, the inputs to all quantized layers add up to roughly 190,000 activation values. A calibration set of 512 images therefore produces about 100 million values to histogram. That is the workload of Algorithm 1.

What gets quantized where: the input of every conv and of the FC layer (activations, INT8, one scale each), and every conv and FC weight matrix (INT4, one scale per output channel). The first conv's input is the image itself and the final logits stay FP32, as is standard.

Self-check

1. A conv has C_in = 32, k = 3, C_out = 64. What are the shapes of W and of one row of X?
2. Why can weights be quantized without any data, but activations can't?
3. Why fold BatchNorm before choosing weight scales, not after?

Lesson 3: Histograms and the clipping trade-off

All three calibration methods work on a histogram of |activation| values instead of the raw data. A histogram turns about 100 million numbers into 2,048 counts, so searching for the best threshold costs the same no matter how much calibration data you use.

What a distribution looks like

A **distribution** describes how often each value occurs. Activations after a ReLU have a characteristic shape: a tall spike at exactly 0 (everything the ReLU zeroed), most values small, and a long thin tail of rare large values. Weights look different: roughly a bell curve centered on 0.

The long tail is the problem. The maximum is set by a handful of extreme values, but almost all the information is in the dense region near zero.

Building a histogram

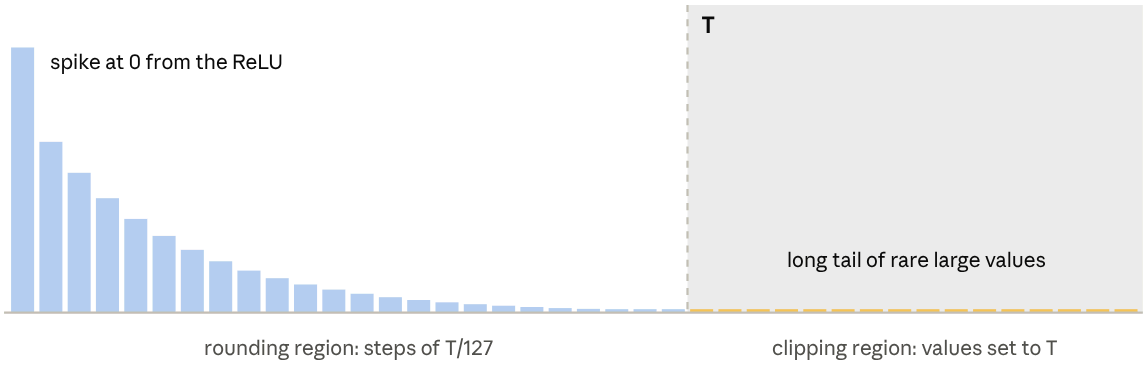
1. Find the largest magnitude, m = max |x|, over all calibration values for the layer.
2. Split [0, m] into B = 2048 equal **bins** of width Δ = m / B.
3. For each value, compute its bin, i = min(⌊|x| / Δ⌋, B − 1), and add 1 to count[i].
4. Divide each count by the total to get a probability p_i. The p_i sum to 1.

Step 3 is the expensive part, and it is what you parallelize. Two facts make that possible:

- **Each value is handled independently.** Any thread can bin any value.
- **Histograms add.** The histogram of all the data equals the sum of the histograms of any split of it. That lets each GPU thread block, or each MPI rank, build its own partial histogram, with a merge at the end.

The catch is that many threads may try to increment the same bin at once, and the spike at zero makes that very likely. Lesson 8 covers how atomics resolve this, and Part 2 shows how to keep the cost down.

Most values sit near zero, so the best T is far below the maximum



illustrative |activation| histogram after a ReLU · not real data

Everything left of T is kept but rounded; everything right of T is clipped. Moving T left makes the steps finer and clips more of the tail.

The trade-off, in histogram terms

With the histogram, the expected squared quantization error for a threshold T can be estimated directly. Let c_i be the center of bin i and s = T/127:

```
E(T) ≈ rounding part + clipping part

rounding part = sum, over bins with c_i ⩽ T, of  p_i × s² / 12
clipping part = sum, over bins with c_i > T, of  p_i × (c_i − T)²

where s = T / 127
```

The first term grows with T, because the step gets coarser. The second shrinks with T, because fewer values are cut. The best T sits where the two balance, and because of the long tail, that is usually far below the maximum.

Example. Suppose 99.99% of a layer's values are below 2.0 but one outlier reaches 20.

- With T = 20, the step is s = 20/127 ≈ 0.157. Every common value is blurred by up to 0.079.
- With T = 2, the step is s ≈ 0.0157, ten times finer for 99.99% of the data. The outlier is clipped from 20 to 2.

Whether the clipped outlier matters depends on the network. That is why there are several methods for choosing T rather than one formula:

Method	How it picks T	Lesson
Max	T = m, no clipping at all. The baseline, and sensitive to outliers	This one
Percentile	The T below which 99.99% (or another fraction) of values fall	5
MSE	The T minimizing E(T) above	5
KL divergence	The T that best preserves the shape of the distribution	4

Self-check

1. m = 8.0 with 2,048 bins. What is Δ, and which bin does 3.0 fall into?

2. Why can two GPU thread blocks build separate histograms and simply add them?
3. In E(T), which term dominates when T is very small, and which when T is very large?

Lesson 4: KL-divergence (entropy) calibration

KL calibration tries every threshold from bin 128 to bin 2048, simulates what INT8 would do to the histogram at each one, and keeps the threshold whose quantized histogram is closest in shape to the original. "Closest" is measured by KL divergence.

The intuition

The network doesn't care about any single activation value. It cares about the overall pattern of values a layer passes on. So instead of measuring per-value error, KL calibration asks a different question: if the next layer saw the quantized distribution instead of the real one, how much information would it lose? That idea comes from information theory, so here is the minimum you need.

Surprise, entropy and KL divergence

- **Surprise.** An event with probability p carries surprise $-\log p$. Rare events (small p) are very surprising; certain events ($p = 1$) carry none.
- **Entropy.** The average surprise of a distribution P, $H(P) = -\Sigma\, p_i \log p_i$. It measures how much information the values carry.
- **KL divergence.** Suppose the data really follows P, but you describe it using a model Q. KL divergence is the average extra surprise you pay for using the wrong model:

```
D_KL(P || Q) = sum over i of  p_i × log( p_i / q_i )
```

Its properties:

- It is always ≥ 0, and it is 0 only when P = Q.
- It is **not symmetric**: $D(P\|Q) \neq D(Q\|P)$. P is always the reference (the original data).
- Terms with $p_i = 0$ contribute nothing, so skip them.
- If $q_i = 0$ where $p_i > 0$, the term is infinite: the model says "impossible" about something that happens. Implementations guard against this with a tiny ε.

Tiny example. P = [0.5, 0.5] and Q = [0.9, 0.1]: $D = 0.5 \cdot \ln(0.5/0.9) + 0.5 \cdot \ln(0.5/0.1) = -0.294 + 0.805 = 0.511$. Q badly underrates the second outcome, and KL charges for it.

The algorithm, step by step

Start with the 2048-bin histogram hist[0..2047] of |x| from Lesson 3, with bin width Δ. INT8's positive half has 128 levels. For each candidate i from 128 to 2048 (1,921 candidates), where clipping at bin i means T = i·Δ:

1. **Reference P.** Copy hist[0..i−1]. Add every count beyond the cut, Σ hist[i..2047], into the last bin P[i−1]. That is literally what clamping does: everything above T lands at T.
2. **Merge.** Split the in-range bins hist[0..i−1] into 128 consecutive groups of about i/128 bins each. Each group is one INT8 level, and its total count is what that level would receive.
3. **Expand back to Q.** P has i bins but the levels number 128, so they can't be compared directly. Spread each group's total evenly over the bins in that group that were **non-zero** in the original (bins that were empty stay empty). Now Q has i bins too.
4. **Normalize** P and Q to sum to 1. Where $q_i = 0$ but $p_i > 0$, replace q_i with a small ε such as 10^{-4}.
5. **Score.** Compute D(P‖Q).

Pick the i with the smallest divergence. The threshold is T = (i + 0.5)·Δ and the scale is s = T/127.

Q merges bins into levels, then spreads each level back over non-empty bins



The empty bin stays empty in Q. After normalizing both rows, D(P‖Q) = 0.059 for this example (Lesson 4, i = 4).

one KL candidate · the worked example below with i = 4 and 2 levels

One candidate threshold, drawn with the numbers from the worked example below: P keeps every bin, Q is what survives quantization, and KL compares the two.

Worked example (shrunk to fit on paper)

Use 8 bins and only 2 levels instead of 2048 and 128. The histogram is hist = [10, 6, 0, 2, 1, 0, 0, 1], 20 values in total.

Candidate i = 8 (no clipping, the max method)

- P = hist / 20 = [0.5, 0.3, 0, 0.1, 0.05, 0, 0, 0.05].
- Groups of 4: [10, 6, 0, 2] totals 18 over 3 non-zero bins, giving [6, 6, 0, 6]. [1, 0, 0, 1] totals 2 over 2 non-zero bins, giving [1, 0, 0, 1].
- Q = [6, 6, 0, 6, 1, 0, 0, 1] / 20 = [0.3, 0.3, 0, 0.3, 0.05, 0, 0, 0.05].
- D = 0.5·ln(0.5/0.3) + 0.3·ln(1) + 0.1·ln(0.1/0.3) + 0 + 0 = 0.255 − 0.110 = **0.146**.

Candidate i = 4 (clip at the halfway point)

- P: copy [10, 6, 0, 2], then add the clipped counts 1 + 0 + 0 + 1 = 2 into the last bin, giving [10, 6, 0, 4]. Divided by 20: [0.5, 0.3, 0, 0.2].
- Groups of 2 over the in-range bins [10, 6, 0, 2]: [10, 6] totals 16 over 2 bins, giving [8, 8]. [0, 2] totals 2 over 1 non-zero bin, giving [0, 2].
- Q = [8, 8, 0, 2] / 18 = [0.444, 0.444, 0, 0.111].
- D = 0.5·ln(1.125) + 0.3·ln(0.675) + 0.2·ln(1.8) = 0.059 − 0.118 + 0.118 = **0.059**.

Clipping wins. With only 2 levels, spending one whole level on the sparse tail wastes resolution where most of the data lives. The real algorithm makes the same trade with 128 levels across 1,921 candidates.

Why it parallelizes well

- Building the histogram is about 100 million independent bin increments (Lesson 3).
- The 1,921 candidates are independent of each other. Each one is a pass over up to 2,048 bins plus a sum, which makes it a natural fit for one GPU thread block, or one share of the MPI ranks.
- The final pick is an **argmin reduction** over 1,921 scores.

Total search work is about $\Sigma\, i \approx 2048^2/2 \approx 2$ million bin operations per layer: small next to the histogram, but enough to show parallel speedup.

Context. This is the method NVIDIA introduced for TensorRT's INT8 calibration, and NVIDIA's own evaluation found it among the best post-training choices for CNN activations (Wu et al., 2020). TensorRT has since deprecated its entropy calibrator class in favor of explicit quantization (TensorRT 10.1), so present KL as a well-established baseline, not the newest technique.

Self-check

1. Why does step 1 add the clipped counts into the last bin, instead of dropping them?
2. In step 3, what would go wrong if you spread a group's mass over its empty bins too?
3. Recompute the worked example for i = 6 with 2 groups of 3 bins. Does it beat i = 4?

Lesson 5: MSE and percentile calibration

Both variants reuse KL's histogram and change only the search step. That means one extra kernel each, and it turns Algorithm 1 into a four-way comparison (max, percentile, MSE, KL) on time and accuracy.

Percentile calibration

Idea: keep a fixed fraction q of the values (99.99%, say) inside the range, and clip the rest.

1. Compute the **cumulative histogram**: cum[i] = hist[0] + hist[1] + … + hist[i]. This is a **prefix sum** (scan), a basic parallel primitive.
2. Find the first bin where cum[i] / total ⩾ q.
3. Set T to that bin's upper edge, (i + 1)·Δ.

It is cheap, with no search at all, but q has to be chosen by hand. In NVIDIA's evaluation, 99.99% and 99.999% were among the best post-training settings for CNN activations, while 99.9% clipped too aggressively and hurt most networks (Wu et al., 2020). Sweep q in {99.9, 99.99, 99.999, 99.9999}.

MSE calibration

Idea: pick the T that minimizes the average squared error between each value and its quantized version. It directly optimizes the rounding-versus-clipping balance from Lesson 3.

With the histogram, treat every value in bin j as sitting at the bin's center c_j. For a candidate T with scale s = T/127, the quantized value of c_j is $Q_T(c_j) = s \cdot \min(\text{round}(c_j / s), 127)$, so:

```
MSE(T) = sum over j = 0 … 2047 of  p_j × ( c_j − Q_T(c_j) )²
```

Evaluate it for the same 1,921 candidates as KL and take the argmin. Every candidate is a full pass over the 2,048 bins, so the total is about 4 million operations per layer, with exactly the same parallel shape as KL: candidates × bins, then an argmin. Nagel et al.

(2021) recommend MSE-based range setting for weights and most activations in their standard PTQ pipeline.

Side by side

Method	What it optimizes	Search cost per layer	Parallel primitive after the histogram
Max	Nothing: no clipping	None	Max-reduction (needed anyway for the bin width)
Percentile	Keeps a fixed fraction of values	One scan over 2,048 bins	Prefix sum, then search
MSE	Average squared error per value	About 4M operations	Candidates × bins, then argmin
KL	Shape of the distribution (information)	About 2M operations	Candidates × bins, then argmin

All four share the same expensive histogram pass. Your timing plots will therefore show nearly identical histogram times and different search times. Your accuracy table will show which threshold choice ResNet-20 prefers.

Self-check

1. Why does percentile calibration need a prefix sum, and why is a prefix sum harder to parallelize than a plain sum?
2. MSE and KL both search the same candidates. What exactly differs in the work done per candidate?
3. If a layer has no outliers at all, what threshold would each method pick?

Lesson 6: The linear algebra behind GPTQ

GPTQ's goal is to quantize a layer's weights so its outputs on real inputs barely change. Everything it needs to know about those inputs fits in one d×d matrix, $H = 2X^TX$. Its off-diagonal entries say which weights can cover for each other's rounding errors.

The goal, stated precisely

From Lesson 2, a layer computes $Y = X W^T$. Look at one output channel at a time: one row w of W (length d) produces the output column $X w$. Quantizing the row changes it to $\hat{w}$, and the output changes by $X(w - \hat{w})$. GPTQ chooses $\hat{w}$ to make that change as small as possible on the calibration inputs:

```
choose w_hat to minimize   || X·w − X·w_hat ||²

(||v||² = the sum of the squares of v's entries)
```

Two things to notice:

- **Rows are independent.** Each output channel has its own row, and the objective for one row never mentions another. That is where the row-level parallelism in Part 2 comes from.
- **It's not "round each weight to its nearest grid point."** Round-to-nearest (RTN) minimizes $\|w - \hat{w}\|$, the error in the weights. GPTQ minimizes the error in the outputs, which can mean deliberately rounding some weights the "wrong" way.

From X to H

Write the weight error as $\delta = w - \hat{w}$. Then:

```
|| X·δ ||² = δᵀ (XᵀX) δ = ½ × δᵀ H δ

where H = 2 × XᵀX
```

X is huge (32,768 × 576 for a late ResNet-20 layer), but H is only d×d (576 × 576). Once H is built, X can be thrown away. H is called the **Hessian** because it is the matrix of second derivatives of the error with respect to the weights; for a quadratic error like this one, the description is exact.

What H means

- **Diagonal, $H_{jj} = 2\Sigma x_j^2$:** how much input j is used. An error on weight j matters in proportion to H_{jj}. A weight whose input is almost always zero can be rounded freely.
- **Off-diagonal, H_{jk}:** whether inputs j and k move together. This is the key to GPTQ.

Example of compensation. Suppose two inputs are always equal, $x_1 = x_2$. The output is $w_1x_1 + w_2x_2 = (w_1 + w_2)x_1$, so only the sum matters. If rounding pushes w_1 up by 0.3, moving w_2 down by 0.3 cancels the error exactly. If x_1 and x_2 were unrelated, nothing could cancel

it. Real inputs are partly correlated, so H tells GPTQ how much of each error to push onto which other weights.

Building H in parallel

Each row x of X (one input patch) contributes an outer product $x\,x^T$, a d×d matrix, and H is their sum:

```
H = 2 × (sum over r = 1 … n of  x_r · x_r^T)

(each x_r · x_r^T is a d × d matrix)
```

Like histograms, partial sums over any split of the rows simply add. Each GPU thread block, or each MPI rank, can sum its share and the partials are combined at the end. The cost is about $2nd^2$ flops: for n = 32,768 and d = 576, that is about 22 GFLOP for one layer, which is real GPU work. Because H is symmetric, only half of it needs computing; that is exactly what cuBLAS's SYRK routine does.

Positive definite, damping and dead inputs

- **Positive semi-definite.** $\delta^T H \delta$ is a squared length, so it is never negative for any δ. When it is strictly positive for every non-zero δ, H is **positive definite** and can be inverted.
- **Dead inputs break that.** If ReLU makes some input channel always zero, its row and column of H are zero, and H can't be inverted. The reference GPTQ code sets such an H_{jj} to 1 and the corresponding weights to 0.
- **Damping.** To stay numerically safe, add λ to every diagonal entry. GPTQ always uses 1% of the average diagonal value (Frantar et al., 2022): $\lambda = 0.01 \cdot \text{mean}(\text{diag } H)$.

Cholesky decomposition

GPTQ needs entries of H^{-1}. Computing an inverse directly is slow and numerically fragile, so it goes through the **Cholesky decomposition**: every positive definite A can be written as $A = L\,L^T$, where L is lower triangular (zeros above the diagonal). Think of it as a square root for matrices.

2×2 example: A = [[4, 2], [2, 3]] gives L = [[2, 0], [1, √2]]. Check: $L\,L^T$ = [[4, 2], [2, 1 + 2]] = A.

- Cholesky costs about $d^3/3$ flops, about 64 million for d = 576. That's small, so use cuSOLVER (potrf) rather than writing it yourself.
- Triangular matrices are easy to work with: solving with one is a simple forward or backward sweep.

GPTQ's recipe: Cholesky-factor H, use it to form H^{-1} (cuSOLVER potri), then Cholesky-factor H^{-1} itself and keep the **upper** triangular factor U, where $H^{-1} = U^T U$. Lesson 7 explains why U contains exactly the numbers the column sweep needs.

Self-check

1. Why can GPTQ discard X once H is built?
2. A layer has d = 288. How big is H, and roughly how many flops is its Cholesky factorization?
3. Why does a ReLU that always outputs zero for one channel make H impossible to invert?

Lesson 7: GPTQ from first principles

GPTQ quantizes a weight matrix one column at a time, left to right. After each column it nudges every column not yet quantized, so their later values cancel the error just made. Three tricks make this fast: the same column order for every row, precomputing the needed numbers with one Cholesky factorization, and batching the updates 128 columns at a time.

Step 1: quantize one weight, fix up the rest

The idea comes from Optimal Brain Surgeon (Hassibi and Stork, 1993) and its quantization version, OBQ (Frantar and Alistarh, 2022). Take one row w and a set F of weights not yet quantized. Quantize one of them, weight q. Its error is e = w_q − quant(w_q).

Now ask: how should the other weights in F change to keep the output error $\delta^T H \delta$ (Lesson 6) as small as possible? This is a small constrained least-squares problem, and its answer is proportional to column q of the inverse Hessian:

```
error     = w_q − quant(w_q)
δ_F       = − error / [H_F⁻¹]_qq  ×  (column q of H_F⁻¹)
```

In words: take the error, divide by a scalar, and spread it over the other weights in proportion to how strongly they are coupled to weight q in H^{-1}. Then remove q from F and update H^{-1} to match (one step of Gaussian elimination), and repeat.

Done weight-by-weight in the best order, as OBQ does, this is accurate but far too slow for large layers.

Tiny worked example

Use 2 weights and a grid of whole numbers. H = [[2, 1], [1, 2]] (inputs positively correlated), so H^{-1} = (1/3)·[[2, −1], [−1, 2]] = [[0.667, −0.333], [−0.333, 0.667]]. Start from w = [0.6, 0.6].

1. Quantize w_1: 0.6 rounds to 1, so e = 0.6 − 1 = −0.4.
2. Update w_2: −(−0.4) / 0.667 × (−0.333) = −0.2, so w_2 becomes 0.4.
3. Quantize w_2: 0.4 rounds to 0.

GPTQ gives ŵ = [1, 0], where plain rounding (RTN) gives [1, 1]. Compare the output error $\delta^T H \delta$:

- RTN: δ = [−0.4, −0.4], so the error is 0.96. Both weights rounded up and, because the inputs move together, the errors add.
- GPTQ: δ = [−0.4, 0.6], so the error is 0.56, about 42% lower, even though w_2 was rounded "the wrong way."

Trick 1: one fixed order for every row

GPTQ's first observation (Frantar et al., 2022): quantizing columns in a **fixed order** (left to right) works about as well as OBQ's careful per-weight order, especially for large layers. Since H depends only on the inputs, it is the same for every row. With a shared order, the sequence of H^{-1} updates is identical for all rows, so it is computed once and applied to all rows simultaneously.

That is the parallel structure you will exploit:

- **Across rows:** fully parallel. Each row is an independent sweep.
- **Across columns:** strictly sequential. Column j+1 can't be quantized until column j's error has been pushed onto it.

Trick 2: Cholesky replaces the H^{-1} updates

The sweep only ever needs one row of the shrinking H^{-1} at each step: at step j, row j from column j onward. GPTQ shows those rows are exactly the rows of the upper Cholesky factor U of H^{-1} ($H^{-1} = U^T U$), up to a scale factor. So instead of d sequential Gaussian-elimination steps, compute U once with cuSOLVER. The update for column j becomes:

```
err_j       = ( W[:, j] − quant(W[:, j]) ) / U[j, j]
W[:, j+1:] −= err_j × U[j, j+1:]

(for every row r and later column k:  W[r, k] −= err_j[r] × U[j, k])
```

Here err_j is a column vector (one value per row), and the update is an **outer product**: every remaining weight W[r, k] decreases by $err_j[r] \cdot U[j, k]$. The paper also notes this Cholesky form is more numerically stable, which is why it is used even for huge models.

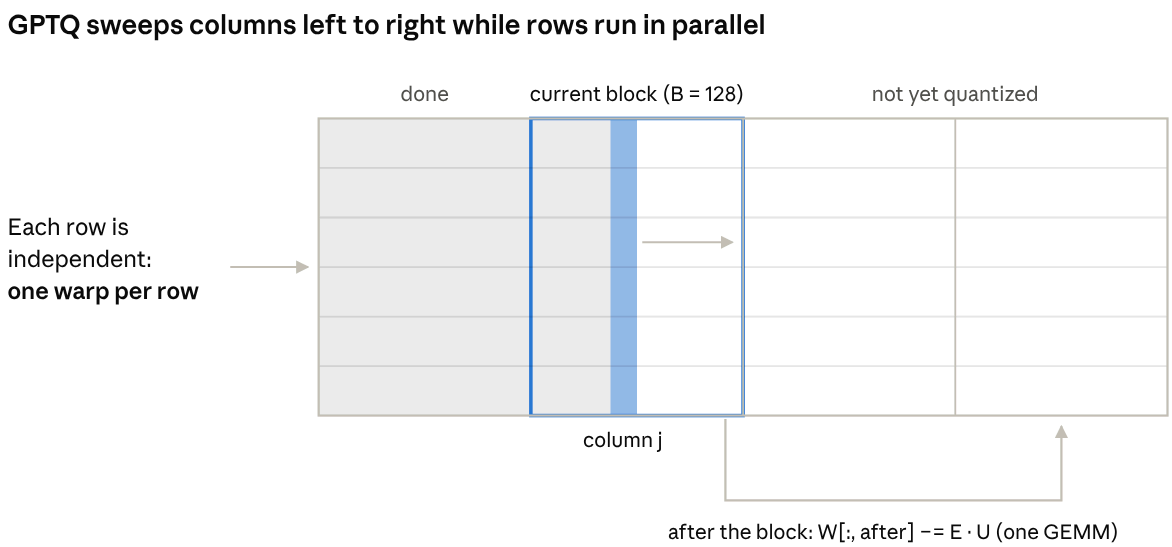
Check against the tiny example: U for that H^{-1} is [[0.816, −0.408], [0, 0.707]]. err_1 = −0.4 / 0.816 = −0.490. Then w_2 −= (−0.490)(−0.408) = 0.2, giving w_2 = 0.4, the same as before.

Trick 3: lazy batch updates

Updating every remaining column after every single column means rereading the whole matrix d times, which is memory-bound and slow. GPTQ instead processes **blocks of B = 128 columns**:

1. Inside the block, do the column-by-column updates only on the block's own 128 columns, and save each err_j into an error matrix E (rows × 128).
2. When the block is done, apply all its errors to every column to the right at once: W[:, after] −= E · U[block rows, after]. That is one big matrix multiply (cuBLAS GEMM).

The math is identical; only the order of the additions changes. For your report this is the clean split: the in-block sweep is **synchronization-heavy** (one barrier per column), while the trailing update is **compute-heavy** (one GEMM per block).



GPTQ column sweep · in-block updates, then one trailing GEMM per block

Inside the current block, quantizing column j updates only the block's later columns; when the block finishes, one matrix multiply pushes its errors onto everything to the right.

The scale grid and act-order

- **Scales are fixed before the sweep.** For INT4 with one scale per row, set $s_r = \max|w_r| / 7$ from the original row. The sweep only decides how each weight rounds and how errors propagate. (Searching each row's scale by MSE is an optional extension.)
- **Act-order.** The GPTQ reference code adds an option to process columns in order of decreasing H_{jj}, so the most-used inputs are quantized first while the most freedom to compensate remains. Implementation: sort the diagonal, permute W's columns and H's rows and columns, run the sweep, then undo the permutation. That is a sort plus a gather, two more parallel steps.

The whole algorithm for one layer

1. Build $H = 2X^TX$ from the calibration inputs (im2col first for convs).
2. Handle dead inputs, then add damping $\lambda = 0.01 \cdot \text{mean}(\text{diag } H)$.
3. Optional act-order: permute.
4. Cholesky of H, then H^{-1}, then the upper Cholesky factor U of H^{-1}.
5. Set per-row scales.
6. For each block of 128 columns: in-block sweep, then the trailing GEMM update.
7. Undo the permutation and write the INT4 values plus scales.

Cost per layer, with d_row rows and d columns: building H costs about $2nd^2$, the factorizations cost $O(d^3)$, and the sweep costs about $d_row \cdot d^2$. The sweep is parallel across d_row rows but has d sequential steps.

Self-check

1. In the tiny example, what would happen if H were [[2, 0], [0, 2]], with independent inputs?
2. Why is it safe to process all rows in parallel, but not all columns?
3. Lazy batching doesn't change the result. What does it change, and why does that matter on a GPU?

Lesson 8: GPU and MPI primer

The GPU programs get speed from thousands of threads sharing fast on-chip memory, and they pay for contention when threads hit the same address. The MPI programs get speed from separate processes with private memory, and they pay for every message sent. Your report has to measure both costs.

How a CUDA program runs

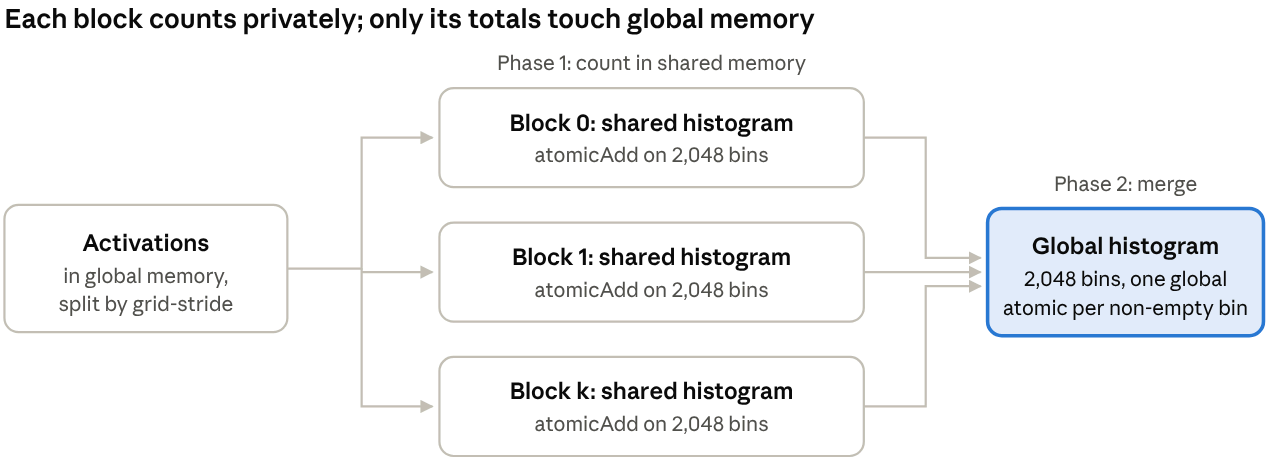
- **Kernel:** a function that runs on the GPU, launched over a **grid** of **thread blocks**: `kernel<<<numBlocks, threadsPerBlock>>>(args)`.
- **Thread block:** up to 1,024 threads that run on the same streaming multiprocessor (SM). They can share **shared memory** (fast, on-chip, tens of KB per block) and wait for each other with `__syncthreads()`.
- **Warp:** 32 threads of a block that execute in lockstep. Threads in a warp can exchange registers directly with shuffle instructions (`__shfl_down_sync`), which is the fastest way to combine values.
- **Blocks can't wait for each other** inside a kernel. The only global barrier is the end of the kernel: launch a second kernel to use all the blocks' results.
- **Memory, fastest to slowest:** registers, shared memory, L2 cache, global memory. Neighboring threads reading neighboring addresses is called **coalesced** access, and it is the cheapest way to read global memory.

Atomics and contention

When many threads increment the same counter, `atomicAdd` guarantees none of the increments are lost. The price: conflicting atomics on one address are **serialized**, one after another. That is **write contention**, and it is exactly what your report's section on read/write contention asks about.

- A histogram is the textbook case. The bin each value lands in is unknown until it's read, so collisions can't be avoided. The worst case is when every value lands in the same bin, and the ReLU spike at zero comes close to that.
- The standard fix is **privatization**: each thread block builds its own histogram in shared memory, then a second phase merges the per-block histograms. NVIDIA measured shared-memory atomics at roughly 2× faster than global atomics on Maxwell-generation GPUs (Sakharnykh, 2015).

- **Read contention** is milder. Many threads reading one address is a broadcast, which is cheap. The cost comes from shared-memory **bank conflicts**, when threads in a warp hit different addresses in the same memory bank.



Colliding atomics on one bin run one at a time; privatizing keeps those collisions in fast shared memory.

two-phase privatized histogram

The two-phase histogram used in P1: contention is confined to each block's shared memory, and global memory sees only one add per non-empty bin per block.

Reductions, argmin and prefix sums

- **Reduction** (sum, max): combine pairs in a tree, so n values take $\log_2 n$ steps. Typically each warp reduces with shuffles, the warp results go through shared memory, and a second kernel (or one atomic per block) combines the blocks.
- **Argmin:** a reduction over (value, index) pairs that keeps the pair with the smaller value. This is how the best threshold is chosen.
- **Prefix sum (scan):** every output is the sum of all earlier inputs. It still takes about log n parallel steps, but needs two passes (up-sweep and down-sweep). Percentile calibration uses one.
- The **CUB** library provides tuned versions of all of these (`BlockReduce`, `DeviceReduce`, `BlockScan`, `DeviceHistogram`). Write your own for the core algorithm and use CUB as the baseline to compare against.

How an MPI program runs

- p **processes** (ranks) each run the same program on their own slice of the data, with **private memory**. Nothing is shared; data moves only through messages.

- **Collectives** used in this project:
  - `MPI_Allreduce`: every rank contributes an array; every rank receives the element-wise sum. Used to merge histograms and partial Hessians.
  - `MPI_Allreduce` with `MPI_MINLOC`: a distributed argmin over (value, rank or index) pairs.
  - `MPI_Bcast`, `MPI_Scatterv`, `MPI_Gatherv`: send one array to all ranks, split one array across ranks, or collect pieces onto one rank.
- **Cost model:** a message of m words costs t_s + m·t_w, where t_s is startup latency and t_w is time per word. A tree-based allreduce takes about $\log_2 p$ rounds. For long messages, good MPI libraries switch to algorithms whose cost is about 2m·t_w, nearly independent of p.

Measuring parallel performance

- **Speedup:** S(p) = T(1) / T(p), where T(1) is the time of the best sequential version.
- **Efficiency:** E(p) = S(p) / p. A value of 1.0 is perfect.
- **Amdahl's law:** if a fraction f of the work can't be parallelized, then $S(p) \leqslant 1 / (f + (1 - f)/p)$. Even 5% serial work caps speedup at 20×. In GPTQ, the sequential column dependency plays this role.
- **The split your report needs:** T(p) = T_compute + T_communication (or T_synchronization on the GPU). Time each phase separately in code, so you can plot them as stacked bars.

Self-check

1. Why can't two thread blocks in the same kernel safely wait on each other?
2. A histogram has one bin receiving 90% of the values. What happens to atomicAdd throughput, and how does privatization help?
3. If 10% of GPTQ's runtime is the sequential sweep, what is the best possible speedup on 64 processes?

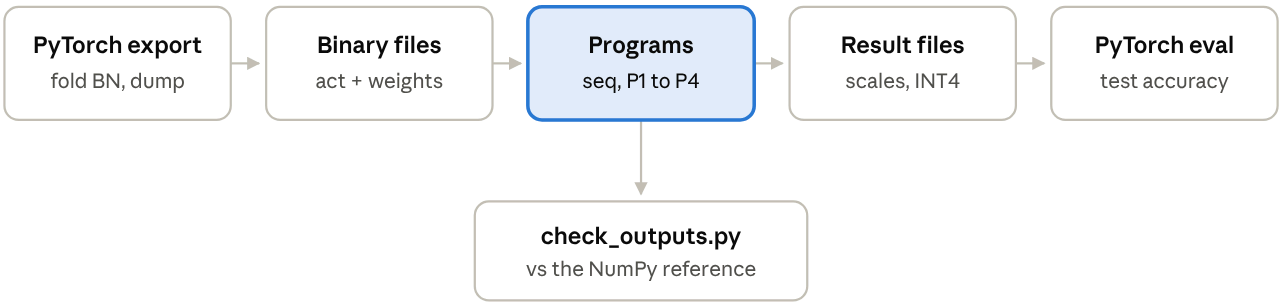
Part 2: Implementation

Implementation overview

A small PyTorch harness does everything that isn't the subject of the report: training, dumping data, and measuring accuracy. The four timed programs read and write plain binary files, so they stay standalone C++/CUDA/MPI and every program sees identical

inputs.

Plain binary files keep the timed programs standalone and comparable



Timing runs start only after every program's output matches the reference.

data pipeline · export, programs, check, evaluate

Python handles the model at both ends; everything timed sits in the middle and reads and writes plain files.

Components

Component	Language	What it does
Harness: export	Python, PyTorch	Train or load ResNet-20, fold BatchNorm, run 512 calibration images, and save each quantized layer's input tensor and weight matrix
Reference implementations	Python, NumPy	Slow, obviously correct versions of KL, MSE and percentile calibration and of GPTQ, for checking outputs
Sequential baselines	C++	Single-threaded versions of both algorithms. These give T(1) for every speedup plot
P1, P3	CUDA C++ (plus cuBLAS, cuSOLVER, CUB)	The GPU programs
P2, P4	C++ with MPI	The distributed programs

Component	Language	What it does
Harness: evaluate	Python, PyTorch	Load scales and quantized weights, apply fake quantization, report CIFAR-10 test accuracy

File format

Keep it trivial so C++ and Python agree:

- **Tensor file:** a header of int32 values (number of dimensions, then each dimension), followed by raw little-endian float32 data. NumPy writes it with `tofile`, and C++ reads it with one `fread`.
- **Per layer:** `layerNN_act.bin` (input activations, N×C×H×W) and `layerNN_w.bin` (folded weights, C_out × C_in × k × k).
- **Outputs:** `act_scales.bin` (one float per layer, per method), and `layerNN_wq.bin` (INT4 values stored as int8) plus `layerNN_wscale.bin` (one float per row).

Size check: the activations of all quantized layers for 512 images total about 100 million floats, roughly 400 MB, which fits in GPU memory at once. Don't dump im2col matrices: they are 9× larger. Do im2col on the GPU (P3) or per rank (P4).

One design decision to state in the report

GPTQ is usually applied layer by layer, with each layer's calibration inputs coming from the already-quantized earlier layers. That makes layers depend on each other. This project uses the simpler variant: every layer's inputs come from the FP32 model, so all layers are independent. That keeps the parallel analysis per layer clean. Mention the sequential variant as a possible accuracy improvement in your conclusions.

P1: Algorithm 1 in CUDA

P1 is five kernels per layer: abs-max, histogram, then one search kernel per method (KL, MSE, percentile), then argmin. The histogram kernel is where your contention analysis lives; the search kernels are where your reduction analysis lives.

Data structures

Name	Type and size	Where	Purpose
`act`	float[n], about 2^{21} to 2^{23} values per layer for 512 images	Global	One layer's activations, flattened
`d_max`	float (stored as int bits)	Global	Result of the abs-max reduction
`s_hist`	uint32[2048], 8 KB	Shared, one per block	Privatized histogram
`d_hist`	uint32[2048]	Global	Merged histogram
`d_score`	float[1921]	Global	KL or MSE score per candidate
`s_gsum`, `s_gnz`	float[128], int[128]	Shared, one per search block	Group totals and non-zero counts for one candidate

Kernel 1: abs-max

A grid-stride loop, then a warp reduction with shuffles, then one atomic per warp. For non-negative floats, comparing the raw bits as integers gives the same order as comparing the floats, so `atomicMax` on an int works.

```
__global__ void absmax_kernel(const float* x, size_t n, int* d_max_bits) {
  float m = 0.f;
  for (size_t i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
       i += (size_t)gridDim.x * blockDim.x)
    m = fmaxf(m, fabsf(x[i]));
  for (int off = 16; off > 0; off >>= 1)                    // warp reduction
    m = fmaxf(m, __shfl_down_sync(0xffffffff, m, off));
  if ((threadIdx.x & 31) == 0)
    atomicMax(d_max_bits, __float_as_int(m));               // valid since m
>= 0
}
```

Kernel 2: privatized histogram

Phase 1 counts into shared memory; phase 2 adds each block's non-zero bins into the global histogram.

```
#define NBINS 2048
__global__ void hist_kernel(const float* x, size_t n, float inv_width,
                            unsigned* d_hist) {
  __shared__ unsigned s_hist[NBINS];
  for (int b = threadIdx.x; b < NBINS; b += blockDim.x) s_hist[b] = 0;
  __syncthreads();

  for (size_t i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
       i += (size_t)gridDim.x * blockDim.x) {
    float a = fabsf(x[i]);
    // Warp-aggregated count for exact zeros (the ReLU spike):
    unsigned zero_mask = __ballot_sync(__activemask(), a == 0.f);
    if (a == 0.f) {
      int leader = __ffs(zero_mask) - 1;
      if ((threadIdx.x & 31) == leader) atomicAdd(&s_hist[0],
__popc(zero_mask));
    } else {
      int bin = min((int)(a * inv_width), NBINS - 1);
      atomicAdd(&s_hist[bin], 1u);                              // shared
atomic
    }
  }
  __syncthreads();
  for (int b = threadIdx.x; b < NBINS; b += blockDim.x)
    if (s_hist[b]) atomicAdd(&d_hist[b], s_hist[b]);          // global merge
}
```

Two details are specific to this problem:

- **The zero spike.** After a ReLU, often half or more of all values are exactly 0. Without special handling, every thread hammers `s_hist[0]`. The `__ballot_sync` trick counts a warp's zeros and issues **one** atomic instead of up to 32. Measure the kernel with and without it; that comparison is a ready-made result for your contention section.
- **The `__activemask()` caveat.** The ballot happens inside a grid-stride loop, so in the last iteration some lanes may have exited. `__activemask()` covers that, and the leader is chosen from the zero lanes only.

Variants to time against each other: (a) global atomics only, no shared histogram, (b) shared privatized, (c) shared + warp-aggregated zeros, and (d) `cub::DeviceHistogram::HistogramEven` as the library baseline.

Kernel 3: KL search, one block per candidate

Grid = 1,921 blocks (candidate i = 128 + blockIdx.x), 256 threads each. Each block:

1. Computes the outlier mass Σ hist[i..2047] and the in-range total with a block reduction.
2. Has threads 0 to 127 each own one group: sum the group's bins and count the non-zero ones. Bin j belongs to group ⌊j·128 / i⌋.
3. Has all threads loop over bins j < i, compute p and q, and accumulate the KL terms.
4. Block-reduces the sum and writes `d_score[blockIdx.x]`.

```
__global__ void kl_kernel(const unsigned* d_hist, float total, float*
d_score) {
  const int i = 128 + blockIdx.x;                // candidate: clip after
bin i-1
  __shared__ float s_gsum[128];
  __shared__ int   s_gnz[128];
  __shared__ float s_red[32];

  // 1. outliers = sum of bins >= i
  float part = 0.f;
  for (int j = i + threadIdx.x; j < NBINS; j += blockDim.x) part +=
d_hist[j];
  float outliers = block_reduce_sum(part, s_red);    // helper: warp shuffles
+ shared

  // 2. group totals (thread g owns group g)
  if (threadIdx.x < 128) {
    int g = threadIdx.x;
    int lo = (g * i + 127) / 128, hi = ((g + 1) * i + 127) / 128;   // ceil
bounds
    float s = 0.f; int nz = 0;
    for (int j = lo; j < hi; ++j) { s += d_hist[j]; nz += (d_hist[j] != 0); }
    s_gsum[g] = s; s_gnz[g] = nz;
  }
  __syncthreads();
  float q_total = block_reduce_sum(threadIdx.x < 128 ? s_gsum[threadIdx.x] :
0.f, s_red);
```

```
  // 3. KL terms
  const float eps = 1e-4f;
  float kl = 0.f;
  for (int j = threadIdx.x; j < i; j += blockDim.x) {
    float p = d_hist[j] + (j == i - 1 ? outliers : 0.f);
    if (p == 0.f) continue;
    p /= total;
    int g = (j * 128) / i;
    float q = d_hist[j] ? s_gsum[g] / s_gnz[g] / q_total : 0.f;
    kl += p * logf(p / fmaxf(q, eps));
  }
  kl = block_reduce_sum(kl, s_red);
  if (threadIdx.x == 0) d_score[blockIdx.x] = kl;
}
```

Check this against the Python reference first. The ceiling bounds in step 2 must place each bin j in exactly group ⌊j·128/i⌋; an off-by-one here silently shifts thresholds. Note one simplification versus some reference implementations: a clipped edge bin that is empty in the in-range histogram gets q = ε rather than its group's mass. Match whichever choice your Python reference makes.

Contention in this kernel: reads only. All 1,921 blocks read the same 8 KB histogram, which is broadcast from L2 and cheap. Loading it into shared memory first is an optimization to measure. Synchronization: three block-level barriers per candidate, no atomics, and no cross-block communication.

Kernel 4: MSE search

The same grid. Each thread loops over all 2,048 bins:

```
float T = i * width, s = T / 127.f, err = 0.f;
for (int j = threadIdx.x; j < NBINS; j += blockDim.x) {
  float c  = (j + 0.5f) * width;
  float qv = s * fminf(rintf(c / s), 127.f);
  err += d_hist[j] * (c - qv) * (c - qv);
}
err = block_reduce_sum(err, s_red) / total;
```

Kernel 5: percentile, and the argmin

- **Percentile:** an inclusive prefix sum of the 2,048 bins (one block with `cub::BlockScan`, or `cub::DeviceScan::InclusiveSum`), then the first bin whose cumulative fraction reaches q.
- **Argmin:** `cub::DeviceReduce::ArgMin` over `d_score`, or your own single-block reduction on (score, index) pairs. The winning index gives T = (128 + idx + 0.5)·width.

Host loop, and what "number of processes" means on the GPU

```
for (int L = 0; L < num_layers; ++L) {
  cudaMemset(d_max_bits, 0, sizeof(int)); cudaMemset(d_hist, 0, NBINS *
sizeof(unsigned));
  absmax_kernel<<<grid, 256>>>(act[L], n[L], d_max_bits);
  float m = read_max(d_max_bits);                       // small device-to-
host copy
  hist_kernel<<<grid, 256>>>(act[L], n[L], NBINS / m, d_hist);
  kl_kernel<<<1921, 256>>>(d_hist, (float)n[L], d_score);
  mse_kernel<<<1921, 256>>>(...); percentile_kernel<<<1, 1024>>>(...);
  argmin(...);                                          // -> scale for layer
L
}
```

For the speedup plots, vary `grid` (the number of thread blocks) for the abs-max and histogram kernels: 1, 2, 4, … up to (SM count × resident blocks per SM). The grid-stride loops keep results correct at any grid size. Time each kernel with `cudaEventRecord` pairs. A later optimization: process all layers in one launch with the layer index in `blockIdx.y`, which makes the GPU's parallelism less limited by small layers.

P2: Algorithm 1 with MPI

Each rank takes a share of the calibration images, builds a local histogram, and one `MPI_Allreduce` sums them. The ranks then split the candidate thresholds and agree on the winner with a second allreduce. Communication per layer is three small collectives, so P2 should scale until the histogram's 2,048-word allreduce starts to dominate.

Data distribution

- **Calibration images:** rank r owns images [r·N/p, (r+1)·N/p). The activation files are stored image-major (N×C×H×W), so each rank's share is one contiguous slice. Read it with `MPI_File_read_at` and no communication.
- **Candidates:** split **cyclically**, so candidate c goes to rank c mod p. KL's cost per candidate grows with i, so a block split would give the last rank the most expensive candidates. A cyclic split balances the load. This choice is worth a sentence in the report.
- **Percentile:** computed redundantly on every rank. Scanning 2,048 bins is cheaper than any message.

Per-layer communication

Step	Call	Message size	Pattern
Global max	`MPI_Allreduce(MPI_MAX)`	1 float	Tree or recursive doubling, $\log_2 p$ rounds
Merge histograms	`MPI_Allreduce(MPI_SUM)`	2,048 × uint64 = 16 KB	Same, or a ring for long messages
Pick the best threshold	`MPI_Allreduce(MPI_MINLOC)` on (score, index)	1 pair per method	$\log_2 p$ rounds

An optimization to measure: batch every layer's histogram into a single allreduce of L × 2,048 words. That pays the latency t_s once instead of L times.

Code sketch

```
// Per layer L, on every rank
float local_max = 0.f;
for (size_t k = 0; k < my_n; ++k) local_max = std::max(local_max,
std::fabs(my_act[k]));
float gmax;
MPI_Allreduce(&local_max, &gmax, 1, MPI_FLOAT, MPI_MAX, MPI_COMM_WORLD);

std::vector<uint64_t> hist(NBINS, 0), ghist(NBINS);
const float inv_w = NBINS / gmax;
for (size_t k = 0; k < my_n; ++k)
  ++hist[std::min((int)(std::fabs(my_act[k]) * inv_w), NBINS - 1)];
MPI_Allreduce(hist.data(), ghist.data(), NBINS, MPI_UINT64_T, MPI_SUM,
MPI_COMM_WORLD);

struct { float score; int idx; } best = {INFINITY, -1}, gbest;
for (int c = rank; c < 1921; c += nprocs) {          // cyclic split
  float s = kl_score(ghist, 128 + c);                // same math as the CUDA
kernel
  if (s < best.score) best = {s, c};
}
MPI_Allreduce(&best, &gbest, 1, MPI_FLOAT_INT, MPI_MINLOC, MPI_COMM_WORLD);
// threshold T = (128 + gbest.idx + 0.5) * gmax / NBINS, identical on every
rank
```

Timing breakdown to record

Wrap each phase in `MPI_Wtime()`: read, local max, the max allreduce, local histogram, the histogram allreduce, local search, and the MINLOC allreduce. Report the maximum over ranks for each phase. Computation is local max + histogram + search; communication is the three allreduces. Their ratio as p grows is the core plot for P2.

P3: Algorithm 2 (GPTQ) in CUDA

P3 has three heavy phases per layer: building H (a large matrix multiply, compute-bound), factorizing it (cuSOLVER, small), and the column sweep (your own kernel, synchronization-bound). The design assigns **one warp per weight row** so the sweep's per-column synchronization happens through warp shuffles instead of block barriers.

Data structures

Name	Type and shape	Where	Purpose
`act`	float[N×C×H×W]	Global	Layer input activations, from the export
`X`	float[n_chunk × d]	Global, reused	im2col of one chunk of images (for example 64 at a time)
`H`	float[d × d]	Global	Hessian, accumulated over chunks
`perm`	int[d]	Global	Act-order permutation
`U`	float[d × d], upper triangle	Global	Upper Cholesky factor of H^{-1}
`W`	float[d_row × d], row-major	Global	Weights, updated in place during the sweep
`scale`	float[d_row]	Global	One INT4 scale per row
`E`	float[d_row × B]	Global	Scaled errors of the current column block
`Q`	int8[d_row × d]	Global	INT4 values, stored one per byte
`s_U`	float[B × B]	Shared, one per block	The U block the sweep reads every column

Step 1: im2col and the Hessian

im2col kernel: one thread per element of X. For output position (img, y, x) and column index c·k·k + ky·k + kx, it reads `act[img][c][y·stride + ky − pad][x·stride + kx − pad]`, or 0 when that falls in the padding. Process images in chunks so X stays small: a stage-1 layer for all 512 images would be about 300 MB.

Hessian accumulation, H += $2X^TX$ per chunk. Two implementations to compare:

1. **Your own tiled kernel:** each thread block computes one 32×32 tile of H, looping over the chunk's rows in steps of 32, staging two 32×32 tiles of X in shared memory. Compute only tiles on or above the diagonal, then mirror. This is the classic shared-memory matrix multiply, and it is a clean subject for the speedup-vs-thread-blocks plot.

2. **cuBLAS baseline:** `cublasSsyrk` with alpha = 2 and beta = 1. Watch out: cuBLAS is **column-major**. A row-major n×d matrix X looks to cuBLAS like a column-major d×n matrix A = X^T, so call SYRK with `CUBLAS_OP_N` on A to get $A \cdot A^T = X^T X$.

A stretch goal worth a paragraph in the report: fuse im2col into the Hessian kernel, loading patches straight into the shared tiles. That avoids storing X at all (an "implicit GEMM").

Step 2: prepare H

- One small kernel: wherever $H_{jj} = 0$, set $H_{jj} = 1$ and zero column j of W (dead input).
- A reduction for mean(diag H), then add $\lambda = 0.01 \times$ mean to every diagonal entry.
- **Act-order (optional):** sort the diagonal descending with `cub::DeviceRadixSort::SortPairsDescending` (keys = diagonal, values = 0..d−1), then one gather kernel permutes W's columns and H's rows and columns.

Step 3: factorize with cuSOLVER

```
// cuSOLVER is column-major. A row-major symmetric H looks identical in
column-major,
// and LOWER in the column-major view is UPPER in your row-major view.
cusolverDnSpotrf(h, CUBLAS_FILL_MODE_LOWER, d, H, d, work, lwork, info);   //
H = L L^T
cusolverDnSpotri(h, CUBLAS_FILL_MODE_LOWER, d, H, d, work, lwork, info);   //
H <- H^{-1}, one triangle
symmetrize_kernel<<<...>>>(H, d);
// copy that triangle across
cusolverDnSpotrf(h, CUBLAS_FILL_MODE_LOWER, d, H, d, work, lwork, info);   //
H^{-1} = L L^T
// Row-major view of H now holds U = L^T in its upper triangle, with H^{-1} =
U^T U.
// The other triangle still holds old values; the sweep and GEMM never read
it.
```

Query the workspace size first with `cusolverDnSpotrf_bufferSize`, and check `info` after each call (non-zero means the matrix wasn't positive definite: raise the damping). The fill-mode swap between column-major and row-major is easy to get backwards, so don't trust the reasoning alone: verify U against `numpy.linalg.cholesky` on a small matrix before going further.

Step 4: per-row scales

One warp per row reduces max|w|, then s_r = max / 7 (a guard for an all-zero row: use s_r = 1).

Step 5: the column sweep, one warp per row

For each block of B = 128 columns, launch this kernel. Each warp owns one row; each lane holds B/32 = 4 of that row's columns in registers. At column j the owning lane broadcasts its value with a shuffle, every lane computes the same quantized value and error, and each lane updates its own later columns. No `__syncthreads()` is needed inside the sweep.

```
template <int B>
__global__ void gptq_sweep(float* W, int d, int d_row, int col0,
                           const float* U, const float* scale,
                           int8_t* Q, float* E) {
  extern __shared__ float s_U[];                       // B*B floats (64 KB
at B = 128)
  for (int t = threadIdx.x; t < B * B; t += blockDim.x)
    s_U[t] = U[(size_t)(col0 + t / B) * d + col0 + t % B];
  __syncthreads();                                     // the only block
barrier

  const int lane = threadIdx.x & 31;
  const int row  = blockIdx.x * (blockDim.x / 32) + threadIdx.x / 32;
  if (row >= d_row) return;                            // whole warp exits
together

  constexpr int PER = B / 32;
  float w[PER];
  #pragma unroll
  for (int t = 0; t < PER; ++t) w[t] = W[(size_t)row * d + col0 + lane + 32 *
t];
  const float s = scale[row];

  #pragma unroll 4
  for (int j = 0; j < B; ++j) {
    const int owner = j & 31, slot = j >> 5;
    float mine = 0.f;
    #pragma unroll
    for (int t = 0; t < PER; ++t) if (t == slot) mine = w[t];   // keeps w[]
in registers
    const float wj  = __shfl_sync(0xffffffff, mine, owner);     // broadcast
column j
```

```
    const float qi  = fminf(fmaxf(rintf(wj / s), -7.f), 7.f);    // INT4
level
    const float err = (wj - qi * s) / s_U[j * B + j];
    if (lane == owner) {
      Q[(size_t)row * d + col0 + j] = (int8_t)qi;
      E[(size_t)row * B + j] = err;
    }
    #pragma unroll
    for (int t = 0; t < PER; ++t) {
      const int k = lane + 32 * t;
      if (k > j) w[t] -= err * s_U[j * B + k];               // update later
columns
    }
  }
}
```

Notes:

- `s_U` needs 64 KB at B = 128, above the default 48 KB per block. Opt in with `cudaFuncSetAttribute(gptq_sweep<128>, cudaFuncAttributeMaxDynamicSharedMemorySize, 65536)`, or use B = 64 (16 KB) and compare.
- **Comparison variant for the report:** a block-barrier version where threads map to (row, column) pairs and each column step needs two `__syncthreads()`. Timing both isolates the cost of synchronization.
- **Parallelism is only d_row warps.** ResNet-20 has at most 64 rows per layer, which is too few to fill a GPU. Run the scaling experiments on ResNet-18 layer shapes (up to 512 rows, d = 4,608) or synthetic layers with up to 4,096 rows.

Step 6: the trailing update with cuBLAS

After each block, apply its errors to every later column: W[:, after] −= E · U[block, after]. In cuBLAS's column-major view, row-major W (d_row × d) is W^T (d × d_row), so the update becomes W^T[after, :] −= U[block, after]^T · E^T:

```
const float minus1 = -1.f, one = 1.f;
int after = col0 + B, m = d - after;           // columns still to quantize
if (m > 0)
  cublasSgemm(hb, CUBLAS_OP_N, CUBLAS_OP_N,
              m, d_row, B, &minus1,
              U + (size_t)col0 * d + after, d,   // column-major view:
(U[block, after])^T, m x B
              E, B,                              // column-major view: E^T, B
x d_row
              &one,
              W + after, d);                     // column-major view:
W^T[after.., :], m x d_row
```

The operand transposes are the classic source of silent bugs here. Check the result of one block against the NumPy reference before anything else.

Host loop for one layer

```
for (chunk : image_chunks) { im2col<<<...>>>(...); hessian_tiles<<<...>>>
(...); } // or cublasSsyrk
prep_dead_and_damp(...);  if (act_order) permute(...);
cholesky_inverse_cholesky(...);                          // three cuSOLVER
calls + symmetrize
row_scales<<<...>>>(...);
for (int col0 = 0; col0 < d; col0 += B) {
  gptq_sweep<B><<<ceil_div(d_row, WARPS), 32 * WARPS, B * B *
sizeof(float)>>>(...);
  trailing_gemm(col0);
}
if (act_order) unpermute(...);
```

Time each phase with CUDA events. Your stacked bars are then: Hessian (compute), factorization (library), sweep (synchronization-bound) and trailing GEMMs (compute). On the GPU, "number of processes" maps to the number of thread blocks for the Hessian kernel and the number of warps for the sweep.

P4: Algorithm 2 (GPTQ) with MPI

P4 splits the calibration images across ranks to build partial Hessians, sums them with one `MPI_Allreduce`, then splits the **weight rows** across ranks for the sweep. Because rows are independent (Lesson 7), the sequential column loop needs no communication at all,

and the only other message is the final gather.

Phase 1: distributed Hessian

1. Rank r reads its share of images from the activation file, runs im2col locally, and computes $H_r = 2X_r^T X_r$. Use the same blocked loops as the sequential baseline, or OpenBLAS `cblas_ssyrk` for both, so the comparison is fair.
2. Combine: $H = \Sigma H_r$.

Two designs to compare in the report:

Design	Communication	Computation
A: `MPI_Allreduce` of H, then every rank factorizes	One allreduce of d^2 floats	The $O(d^3)$ factorization repeated on every rank (redundant, but in parallel)
B: `MPI_Reduce` to rank 0, rank 0 factorizes, `MPI_Bcast` of U	One reduce + one broadcast of d^2 floats	The factorization runs once, but every other rank waits

Message sizes: d = 576 gives 331,776 floats (1.3 MB). A ResNet-18 layer with d = 4,608 gives 21.2 million floats (85 MB). At that size the Hessian allreduce becomes a real communication cost, which makes P4's scaling plots interesting.

Factorize with LAPACK through LAPACKE (`LAPACKE_spotrf` and `LAPACKE_spotri`). These are library calls, the CPU counterpart of cuSOLVER.

Phase 2: distributed sweep

- **Rows are block-distributed:** rank r owns rows [r·d_row/p, (r+1)·d_row/p), and every rank has the full U.
- Each rank runs the sequential lazy-batch sweep on its rows: the in-block column loop, then a local `cblas_sgemm` for the trailing update.
- **No messages during the sweep.**
- At the end, `MPI_Gatherv` collects the INT4 values (int8) and per-row scales on rank 0.

The rejected alternative, worth explaining in the report: splitting **columns** across ranks would force the rank that owns column j to send its error vector to everyone after every column. That is d sequential broadcasts, each costing about $\log_2 p \cdot t_s$ in latency. Splitting rows removes that dependency entirely, which is the main design argument of P4.

Load-balance caveat: with d_row rows and p ranks, each rank gets d_row/p rows. ResNet-20 layers have at most 64 rows, so beyond p = 64 some ranks would sit idle. Use ResNet-18 shapes (up to 512 rows) for the scaling runs.

Code sketch

```
// Phase 1
std::vector<float> Hl(d * d, 0.f), H(d * d);
for (auto& chunk : my_image_chunks) {
  im2col_cpu(chunk, X);                                    // n_chunk x d
  cblas_ssyrk(CblasRowMajor, CblasUpper, CblasTrans, d, n_chunk,
              2.f, X.data(), d, 1.f, Hl.data(), d);      // Hl += 2 X^T X
}
MPI_Allreduce(Hl.data(), H.data(), d * d, MPI_FLOAT, MPI_SUM,
MPI_COMM_WORLD);  // design A
prep_dead_and_damp(H, W_local);
factorize_to_U(H);                                          // LAPACKE spotrf,
spotri, spotrf

// Phase 2: rows [r0, r1) only
for (int col0 = 0; col0 < d; col0 += B) {
  sweep_block(W_local, U, col0, B, scales, Q_local, E);    // same math as
the CUDA kernel
  trailing_update(W_local, U, E, col0, B);                // cblas_sgemm
}
MPI_Gatherv(Q_local.data(), my_rows * d, MPI_INT8_T,
            Q.data(), counts, displs, MPI_INT8_T, 0, MPI_COMM_WORLD);
```

Timing breakdown to record

Phases: read, im2col + local Hessian (computation), Hessian allreduce (communication), factorization (computation, replicated), sweep (computation) and gather (communication). Plot the stacked bars against p, for design A and design B.

Complexity, contention and synchronization

KL calibration is dominated by one pass over the activations, with tiny messages, so both of its programs should scale well until the per-layer data gets small. GPTQ has a strictly sequential column loop and, in MPI, a d^2-word allreduce, so expect its speedup to flatten earlier. Explaining exactly where and why is the heart of both reports.

Notation. n_a = activations per layer; 2,048 bins; C = 1,921 candidates; n = im2col rows; d = C_in·k·k; d_row = output channels; B = 128; P = concurrent GPU threads; p = MPI ranks; t_s = message latency; t_w = time per 32-bit word.

Parallel time

Program	Computation	Communication or synchronization
P1 KL (GPU)	$O(n_a / P)$ for abs-max and histogram, times a contention factor $\kappa \geqslant 1$ for colliding atomics; $O(C \cdot 2048 / P)$ for the search, with at most C blocks	Two kernel boundaries plus one small device-to-host copy per layer; three to four block barriers per candidate; $\log_2 C$ steps for the argmin
P2 KL (MPI)	$O(2n_a / p)$ for max and histogram + $O(C \cdot 2048 / p)$ for the search	$\log_2 p \cdot (t_s + t_w)$ for the max + $\log_2 p \cdot (t_s + 4096\, t_w)$ for the histogram (2,048 uint64) + $\log_2 p \cdot (t_s + 2t_w)$ per method for MINLOC
P3 GPTQ (GPU)	$2nd^2 / P$ for H; $O(d^3)$ for the cuSOLVER factorizations; about $d_{row} \cdot d^2 / P$ for sweep + GEMMs, with $P \leqslant 32 \cdot d_{row}$ in the sweep	A critical path of d sequential column steps (shuffles inside a warp); d/B sweep launches + d/B GEMM launches; one block barrier per launch
P4 GPTQ (MPI)	$2nd^2 / p$ for the local H + about $(4/3)d^3$ for the factorizations, repeated on every rank (design A) + $d_{row} \cdot d^2 / p$ for the sweep	Design A: about $\log_2 p \cdot t_s + 2d^2\, t_w$ for the allreduce (large-message algorithm); gather: $(p - 1)\, t_s + (d_{row} \cdot d / 4)\, t_w$

What the models predict

P2. Computation shrinks as 1/p while communication grows only as $\log_2 p$, with tiny messages:

```
work        = (2·n_a + 2048·C) × t_c
comm        = log2(p) × (3·t_s + 4099·t_w)

S_P2(p)    ≈ work / ( work / p + comm )
```

(t_c = time per operation.) Near-linear speedup until n_a/p gets small enough that the $\log_2 p$ latency terms matter, which happens first for the 8×8 stage-3 layers.

P4. The replicated factorization is a serial fraction in the Amdahl sense:

```
f = (4/3)·d³ / ( 2·n·d² + (4/3)·d³ + d_row·d² )
```

For a ResNet-20 layer with n = 32,768, d = 576 and d_row = 64, f is under 1.2%, so Amdahl allows a speedup of about 85. The $2d^2$ t_w allreduce does not shrink with p either, though, and for large d it will dominate first. Compute the crossover p with your measured t_s and t_w.

P3. The Hessian kernel should reach a large fraction of the GPU's peak, while the sweep is limited by its d-step critical path. Expect the sweep's share of runtime to grow as the Hessian kernel gets faster.

Contention (shared-memory programs)

Where	Kind	Severity and mitigation to measure
P1 histogram bins	Write: atomicAdd collisions	High, because of the ReLU spike at 0. Shared-memory privatization, then warp-aggregated zero counting. Measure 4 variants
P1 histogram merge	Write: global atomics	Low: 2,048 × grid atomics, mostly non-colliding
P1 search, histogram reads	Read: all blocks read the same 8 KB	Low: broadcast from L2; test staging it in shared memory
P3 Hessian tiles	Read: shared-memory bank conflicts	Medium: pad tiles to 33 columns
P3 sweep, `s_U` reads	Read: all warps read the same row of U	Low: same address across a warp is a broadcast
P3 sweep	Write	None: each warp owns its row exclusively
P1 abs-max	Write: atomicMax	Low: one atomic per warp

Synchronization overheads to report

- **Kernel launch and boundary:** about 5 to 10 μs each. P3 launches 2d/B kernels per layer; measure launch time against kernel time for small layers.

- **Block barriers:** P1′s search uses three to four per candidate. P3′s warp-per-row sweep uses one per launch; the block-barrier variant uses 2B per launch. Timing both is your direct measurement of synchronization cost.
- **Device-to-host synchronization:** P1 copies the max back once per layer. Optional optimization: keep it on the device and compute the bin width in the histogram kernel itself.

Experiments

Every timing point is the median of 10 runs after one discarded warm-up, with min and max shown as error bars. Each program logs its phases separately, so every plot can split computation from communication or synchronization.

Timing method

- **GPU:** `cudaEventRecord` pairs around each kernel and around the whole pipeline. Host-to-device copies of the input are reported separately, not inside compute time.
- **MPI:** `MPI_Barrier`, then `MPI_Wtime` around each phase. Report the maximum over ranks (`MPI_Reduce` with `MPI_MAX`): the slowest rank decides when the job is done.
- **Baselines:** T(1) is the sequential C++ program. Also report speedup relative to the 1-rank MPI run, which makes MPI overhead visible.
- **Record the setup:** GPU model, CUDA version, MPI implementation, CPU model, core count and interconnect. Pin MPI ranks to cores if the cluster allows it.

Run matrix

Program	Varied	Range	Held fixed
P1	Thread blocks (abs-max, histogram)	1, 2, 4, … up to SM count × resident blocks per SM	512 images, 256 threads per block
P1	Threads per block	128, 256, 512, 1024	Full grid
P1	Calibration images	64, 128, 256, 512, 1024	Best configuration
P1	Histogram variant	Global atomics; shared; shared + warp-aggregated zeros; CUB	512 images

Program	Varied	Range	Held fixed
P2	Ranks p, strong scaling	1, 2, 4, 8, 16, 32 (64 if available)	512 images
P2	Ranks p, weak scaling	Same	64 images per rank
P2	Allreduce batching; candidate split	Per layer vs all layers at once; block vs cyclic	p sweep
P3	Thread blocks (Hessian kernel); warps per block (sweep)	1 up to full occupancy; 2, 4, 8, 16	One layer shape at a time
P3	Layer shape	ResNet-20 ($d \leqslant 576$, 64 rows), ResNet-18 ($d \leqslant 4{,}608$, 512 rows), synthetic $4{,}096 \times 4{,}096$	Fixed calibration size
P3	Implementation choices	Own Hessian kernel vs `cublasSsyrk`; warp sweep vs block-barrier sweep; B in {32, 64, 128}	ResNet-18 shapes
P4	Ranks p	1 to 32 (64)	ResNet-18 shapes
P4	Design	A (allreduce + replicated factorization) vs B (reduce, factorize, broadcast)	p sweep

Plots for each program

1. Execution time vs p (MPI) or thread blocks (GPU), on log-log axes, one line per problem size.
2. Speedup vs p with the ideal line and the model curve from the complexity section.
3. Stacked bars per p, splitting computation from communication or synchronization.

Accuracy evaluation (both reports)

All with fake quantization in the PyTorch harness, on the 10,000 CIFAR-10 test images:

Configuration	Weights	Activations	Purpose
Baseline	FP32	FP32	Your reference accuracy
Activation methods	FP32	INT8: max, percentile (99.9 to 99.9999), MSE, KL	Report 1: which calibration ResNet-20 prefers
Weight methods	INT4: RTN, GPTQ, GPTQ + act-order	FP32	Report 2: what error compensation buys
Combined	INT4 GPTQ	INT8, best method	The final W4A8 model

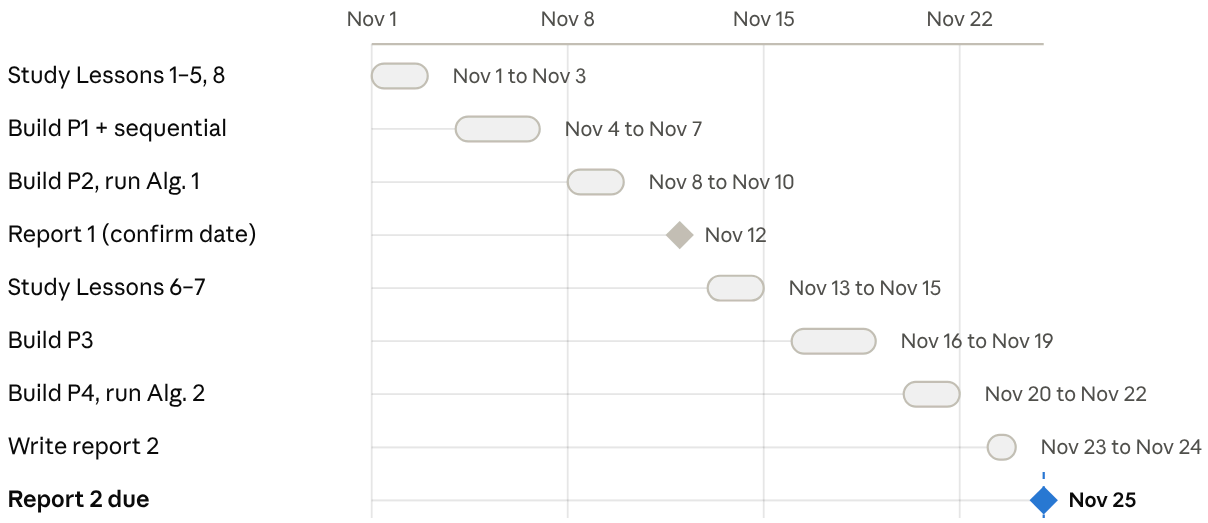
Also log per-layer reconstruction error: the output MSE $\|XW^T - X\hat{W}^T\|^2$ for GPTQ vs RTN. It explains which layers lose accuracy.

Correctness checks (before any timing)

- **KL, MSE and percentile:** every program must choose the same bin index as the Python reference on every layer. Float summation order can differ slightly, so if a tie flips, compare scores to within 10^{-5}.
- **GPTQ:** compare the INT4 output against the reference. Floating-point order differences can flip a few rounding decisions, so require at most 0.1% mismatched values and nearly identical output MSE.
- **Cross-check:** the GPU and MPI versions must agree at every p. Automate this in the run script, since a scaling plot of a wrong answer is worthless.

Study and build plan

The plan builds the slow, obviously correct version of each piece first and checks every faster version against it. Algorithm 1 is finished and reported before you start studying GPTQ, which keeps the first report off the critical path of the harder algorithm.



milestones Nov 1 to Nov 25, 2026

Algorithm 1 is finished and written up before GPTQ study starts; the presentation in the last classes comes after Nov 25 and reuses both reports' plots.

Order of work

1. **Harness and data.** Train or load ResNet-20, fold BatchNorm, measure FP32 accuracy, and export activations and weights for 512 images. Nothing else can be checked until this exists.
2. **Algorithm 1 reference** in NumPy: max, percentile, MSE and KL on one layer's histogram. Re-run the worked example from Lesson 4 as a unit test.
3. **Sequential C++ baseline** for Algorithm 1. It must match the reference bin for bin.
4. **P1:** abs-max → histogram (all 4 variants) → KL → MSE → percentile → argmin. Check against step 3 after each kernel.
5. **P2,** then the Algorithm 1 timing runs, accuracy table and report 1.
6. **GPTQ reference** in NumPy, checked on the 2-weight example from Lesson 7 and against RTN on one real layer.
7. **Sequential C++ GPTQ,** with OpenBLAS and LAPACKE.
8. **P3,** in this order: im2col → Hessian (cuBLAS first, then your own kernel) → cuSOLVER chain (verify U) → sweep → trailing GEMM → act-order.
9. **P4,** then the GPTQ timing runs, the accuracy table, and report 2 with the comparison section.

Repository layout

```
ptq-parallel/
  harness/        export.py  evaluate.py  train_resnet20.py
  reference/      calib_ref.py  gptq_ref.py  test_examples.py
  common/         tensor_io.h  timer.h  args.h
  seq/            calib_seq.cpp  gptq_seq.cpp
  cuda/           calib.cu  gptq.cu  kernels/  (one .cuh per kernel)
  mpi/            calib_mpi.cpp  gptq_mpi.cpp
  scripts/        run_matrix.sh  check_outputs.py  plot.py
  results/        csv/  plots/
  report/
```

- Every program writes one CSV row per run: program, method, p or grid, size, phase, seconds. `plot.py` draws every figure from those rows, so rerunning an experiment regenerates the plots.
- `check_outputs.py` diffs every program's output against the reference, and `run_matrix.sh` calls it before recording any timing.

Mapping to the required report sections

Required section	Where it comes from in this doc
1. Your problem	Overview, plus Lessons 1 and 2 condensed to half a page
2. Chosen algorithms	Lessons 4 and 5 (report 1); Lesson 7 (report 2)
3. Associated data structures	The data-structure tables in P1 and P3, plus the MPI distributions
4. Communication pattern (distributed)	The P2 and P4 per-phase tables, including the rejected column split
5. Read/write contention and synchronization (shared memory)	The contention and synchronization tables, with your histogram-variant and sweep-variant timings as evidence
6. Parallel time complexity	The complexity table and models, with fitted t_s and t_w
7. Timing experiment details	Run matrix and timing method

Required section	Where it comes from in this doc
8. Performance plots	Time, speedup with model curve, stacked bars
9. Conclusions	Where the model and measurements agree or disagree, the limitations (small layers, Amdahl, allreduce size) and future work (fused im2col, sequential-input GPTQ, MSE scale search)

For report 2, add the comparison section: KL vs GPTQ on scalability (communication volume 2,048 words vs d^2), on the kind of parallelism (fully parallel histogram vs sequential sweep), and on accuracy impact.

References

Quantization methods

1. Frantar, Ashkboos, Hoefler and Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323, 2022. Source for the fixed-order insight, lazy batch updates with B = 128, the Cholesky reformulation and 1% damping (Lesson 7, P3).
2. Frantar and Alistarh. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning. NeurIPS 2022. The OBQ method GPTQ builds on.
3. Hassibi and Stork. Second Order Derivatives for Network Pruning: Optimal Brain Surgeon. NeurIPS 1992 (published 1993). The original compensation idea, as cited in reference 1.
4. Nagel, Fournarakis, Amjad, Bondarenko, van Baalen and Blankevoort. A White Paper on Neural Network Quantization. arXiv:2106.08295, 2021. Quantizer formulas, rounding vs clipping error, per-tensor vs per-channel, and MSE range setting (Lessons 1, 3 and 5).
5. Wu, Judd, Zhang, Isaev and Micikevicius. Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation. arXiv:2004.09602, 2020. NVIDIA's comparison of max, entropy and percentile calibration, and the per-channel weight results (Lessons 2, 4 and 5).
6. He, Zhang, Ren and Sun. Deep Residual Learning for Image Recognition. arXiv:1512.03385, 2015. ResNet, including ResNet-20 for CIFAR-10.

7. Benchmarking Post-Training Quantization of Large Language Models under Microscaling Floating Point Formats. arXiv:2601.09555, 2026. Current comparison showing error-compensation methods like GPTQ remain strong; use for the related-work paragraph.

NVIDIA documentation and engineering notes

8. NVIDIA TensorRT. IInt8EntropyCalibrator2 class reference. The entropy calibrator, deprecated since TensorRT 10.1 in favor of explicit quantization.
9. NVIDIA Model Optimizer. model_calib reference. NVIDIA's current calibration algorithms (max, MSE, AWQ, SmoothQuant, SVDQuant and others).
10. Sakharnykh. GPU Pro Tip: Fast Histograms Using Shared Atomics on Maxwell. NVIDIA Developer Blog, March 2015. The two-phase privatized histogram and its contention behavior (Lesson 8, P1).
11. NVIDIA. cuSOLVER documentation. `cusolverDn<t>potrf` and `cusolverDn<t>potri` are listed under the legacy dense API, which is still supported; `cusolverDnXpotrf` is the newer 64-bit interface.
12. NVIDIA. cuBLAS and CUB (CCCL) documentation on docs.nvidia.com, for `cublasSsyrk`, `cublasSgemm`, `BlockReduce`, `DeviceReduce::ArgMin`, `DeviceScan`, `DeviceHistogram` and `DeviceRadixSort`. Look these up for your installed CUDA version.

All pages were checked on Oct 31, 2026, except references 2, 3, 6 and 12, which were identified by search but not opened.